Ensamblado y desensamblado de LEGv8 + Implementación de la ISA

Arquitectura de Computadoras 2026

Arithmetic Operations

- Three operands: two sources and one destination

```
ADD a, b, c     // a gets b + c
```

- C code:

```
f = (g + h) - (i + j); // f-> X19 g->X20, h->X21
                       // i->X22, j->X23
```

- Compiled LEGv8 code:

```
ADD   X0,   X20,   X21          //   X0   =   g   +   h
ADD   X1,   X22,   X23          //   X1   =   i   +   j
SUB X19, X0, X1    // f = X0 - X1
```

LEGv8 R-format Instructions



- Instruction fields
  - opcode: operation code
  - Rm: the second register source operand
  - shamt: shift amount (00000 for now)
  - Rn: the first register source operand
  - Rd: the register destination

R-format Example



ADD X9,X20,X21

1112_{ten}	21_{ten}	0_{ten}	20_{ten}	9_{ten}
10001011000_{two}	10101_{two}	000000_{two}	10100_{two}	01001_{two}

$$1000\ 1011\ 0001\ 0101\ 0000\ 0010\ 1000\ 1001_{two} =$$

$$8B150289_{16}$$

Memory Operands

- Load values from memory into registers
- Store result from register to memory
- Memory is byte addressed

- C code: (elementos de A = DobleWord)

```
A[12] = h + A[8]; // h->X21, base address of A->X22
```

- Compiled LEGv8 code:

```
LDUR    X9, [X22,#64] // Index 8 requires offset of 64
ADD     X9, X21, X9
STUR    X9, [X22,#96]
```

LEGv8 D-format Instructions

opcode	address	op2	Rn	Rt
11 bits	9 bits	2 bits	5 bits	5 bits

- Load/store instructions
  - Rn: base register
  - address: constant offset from contents of base register (-256 to 255)
  - Rt: destination (load) or source (store) register number
  - op2 = “00”

Branch CBZ

Branch to a labeled instruction if a **condition** is true. Otherwise, continue sequentially:

- CBZ register, L1

  if (register == 0) branch to instruction labeled L1;



“address” (n° instructions) is PC-relative

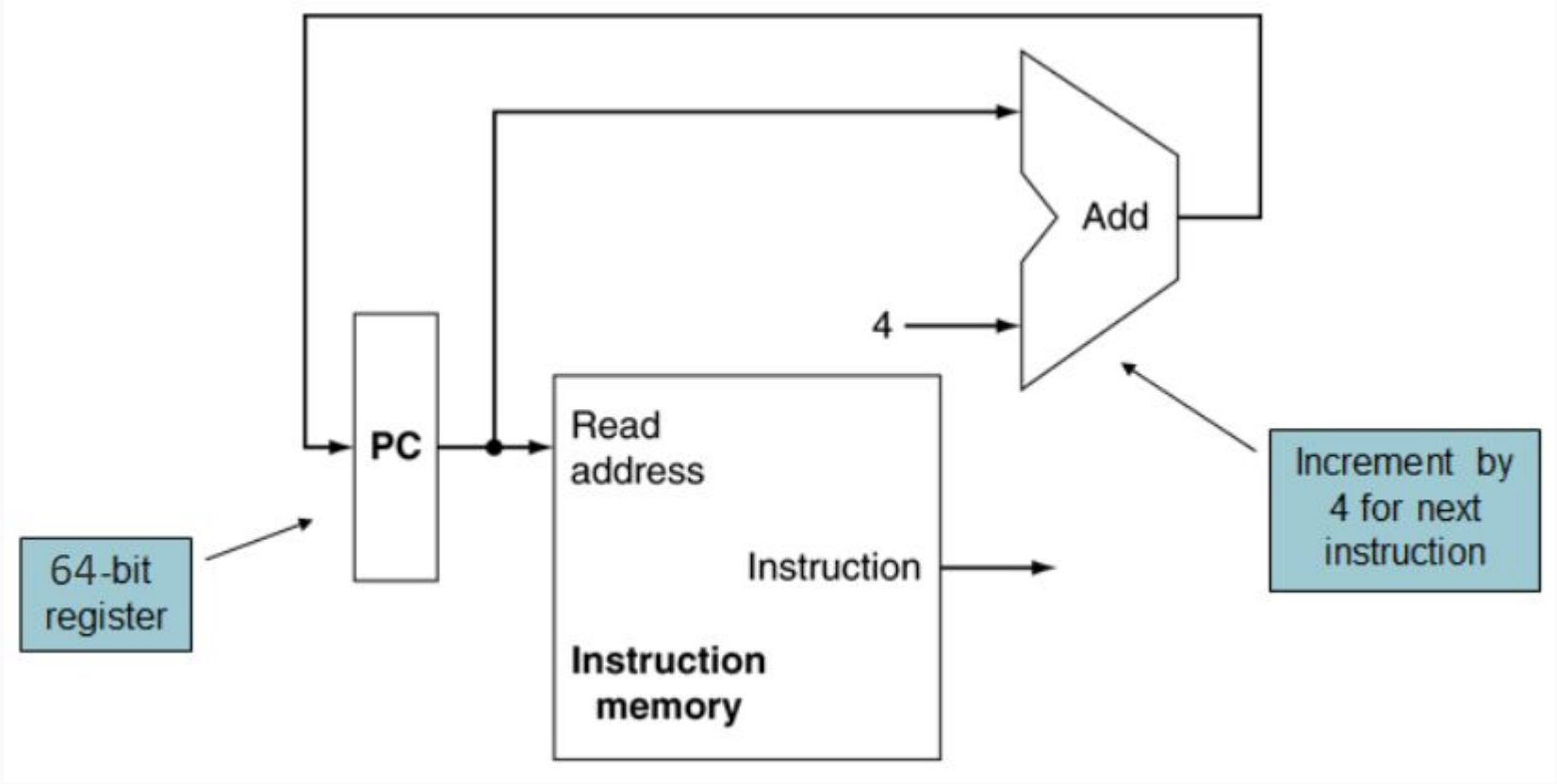
New_PC = Current_PC + (“address” * 4)

Building a Datapath

Datapath

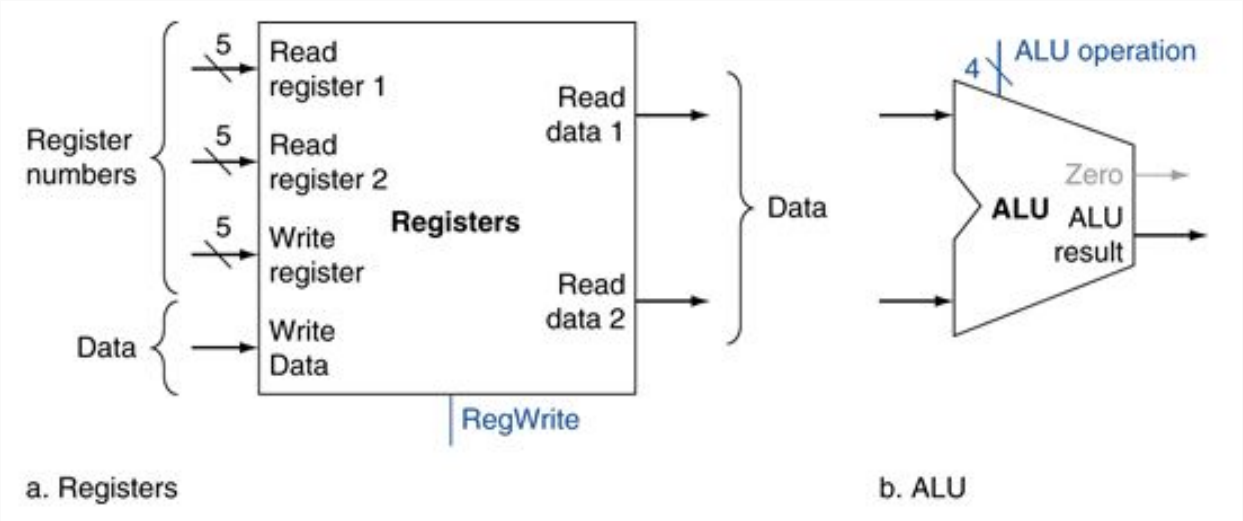
- Elements that process data and addresses in the CPU
  - Registers, ALUs, mux's, memories…
- We will build a LEGv8 datapath incrementally
  - Refining the overview design

Instruction Fetch



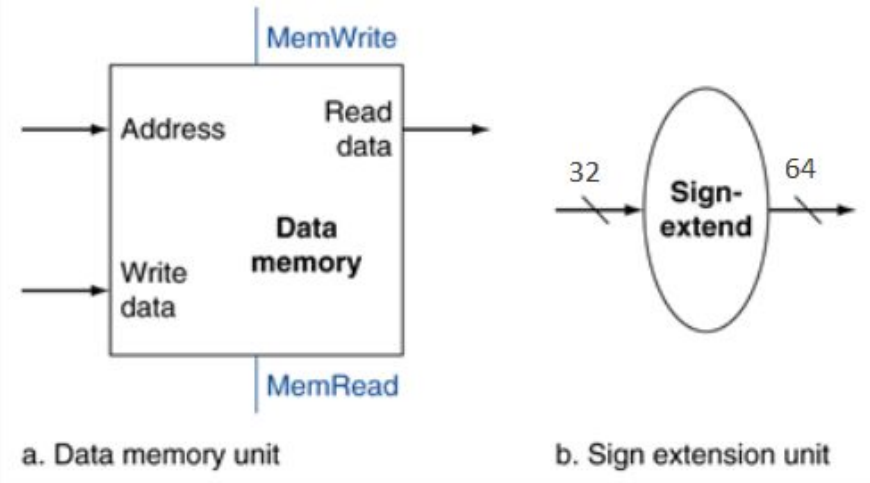
R-Format Instructions

- Read two register operands
- Perform arithmetic/logical operation
- Write register result



Load/Store Instructions (64 bits)

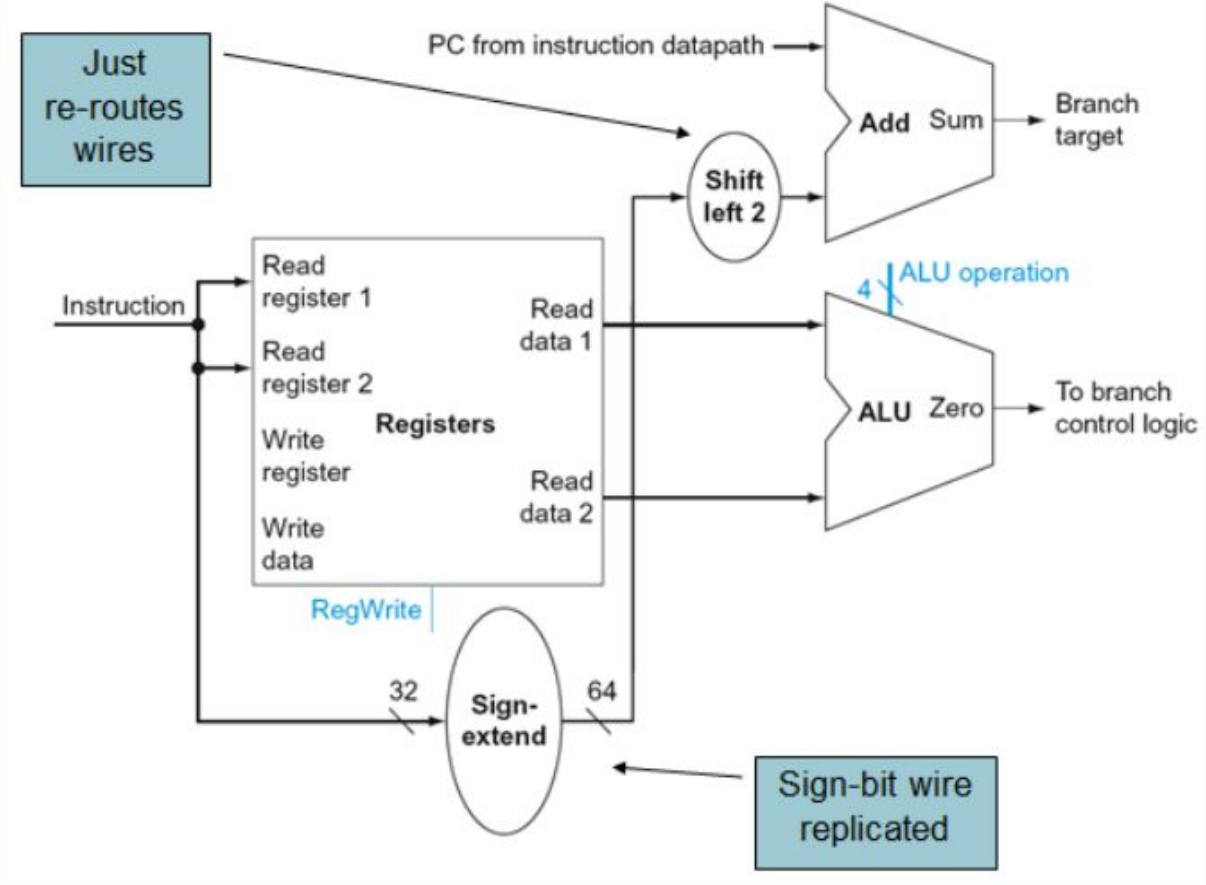
- Read register operands
- Calculate address using 9-bit offset (use ALU)
- Load: Read memory and update register
- Store: Write register value to memory



Branch Instruction (CBZ)

- Read register operand
- Compare operand
  - Use ALU and check Zero output
- Calculate target address
  - Sign-extend displacement
  - Shift left 2 places (word displacement)
- Add to PC

Branch Instruction (CBZ)

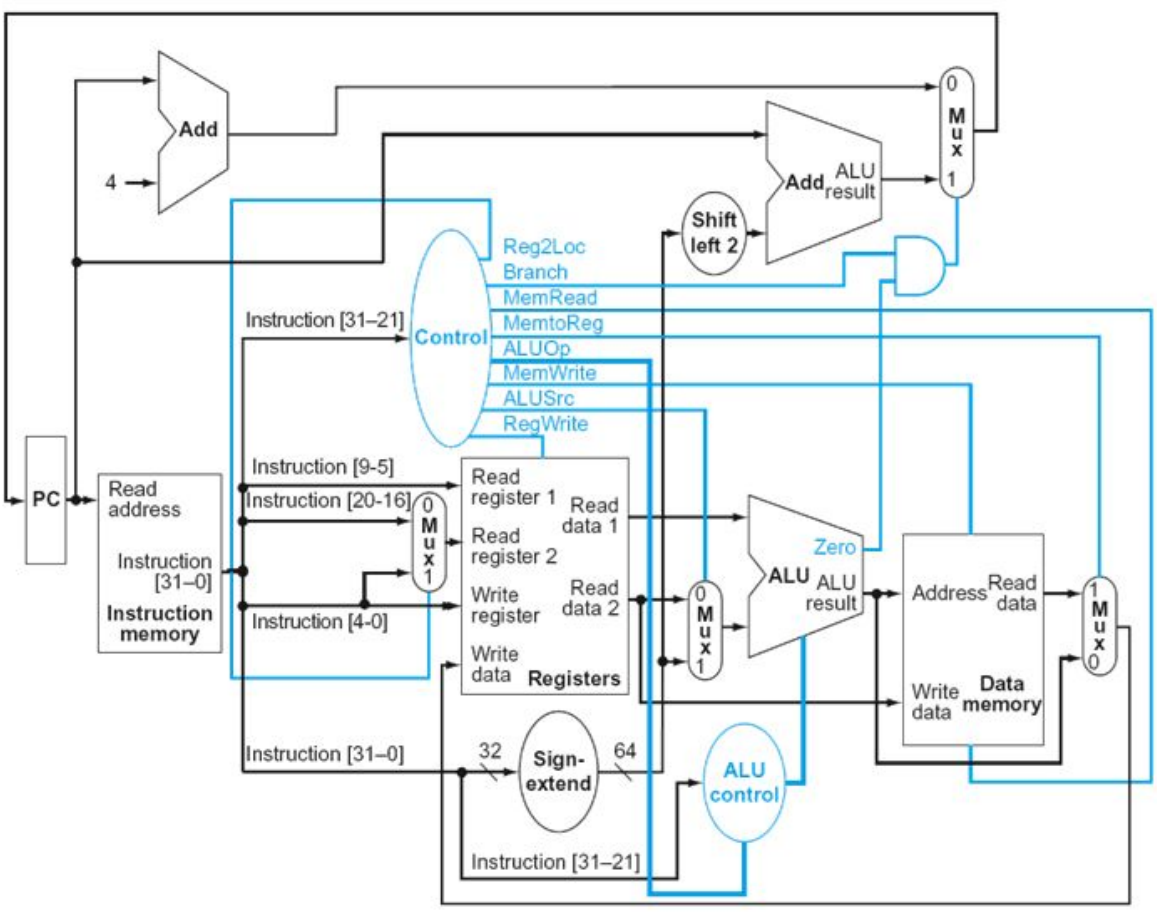


ALU Control

- ALU used for
  - Load/Store: F = add
  - Branch: F = pass input b
  - R-type: F depends on opcode

opcode	ALUOp	Operation	Opcode field	ALU function	ALU control
LDUR	00	load register	XXXXXXXXXXX	add	0010
STUR	00	store register	XXXXXXXXXXX	add	0010
CBZ	01	compare and branch on zero	XXXXXXXXXXX	pass input b	0111
R-type	10	add	100000	add	0010
		subtract	100010	subtract	0110
		AND	100100	AND	0000
		ORR	100101	OR	0001

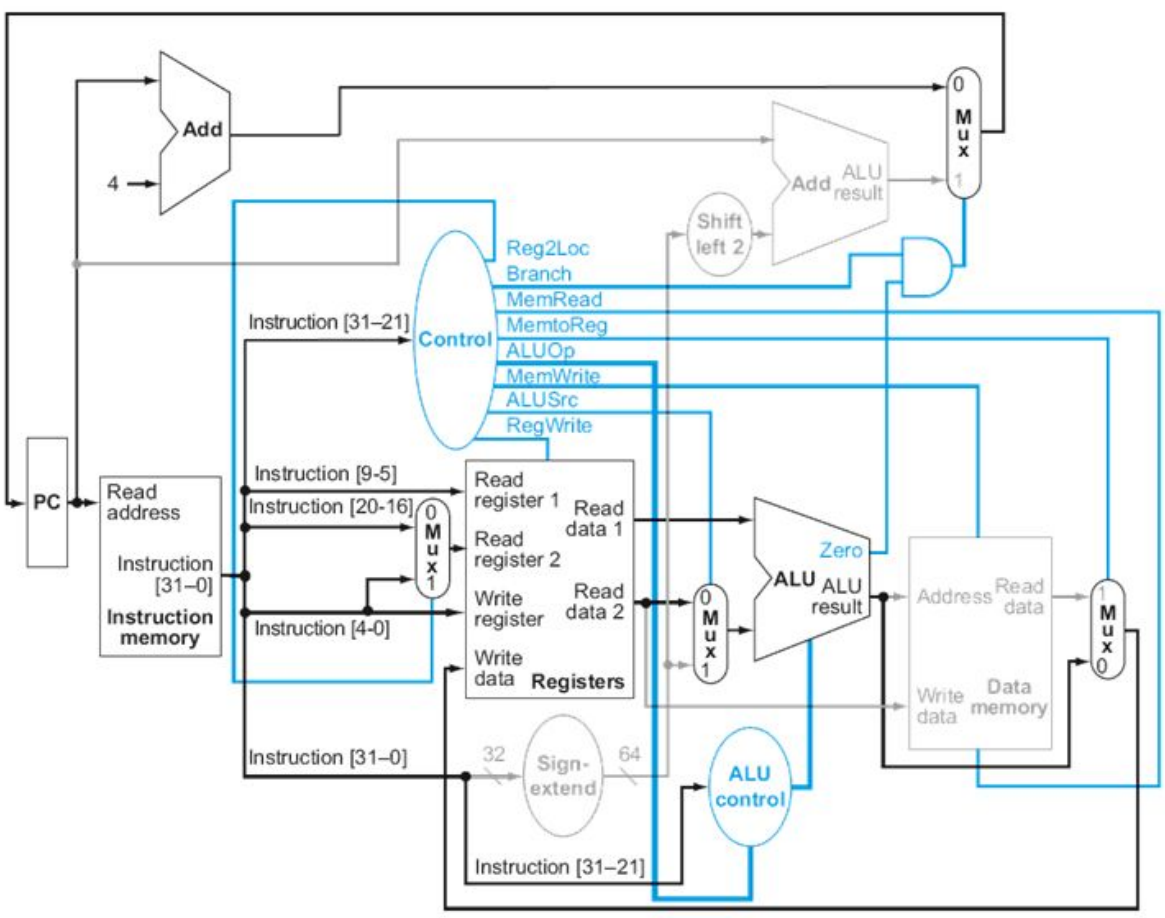
Datapath With Control



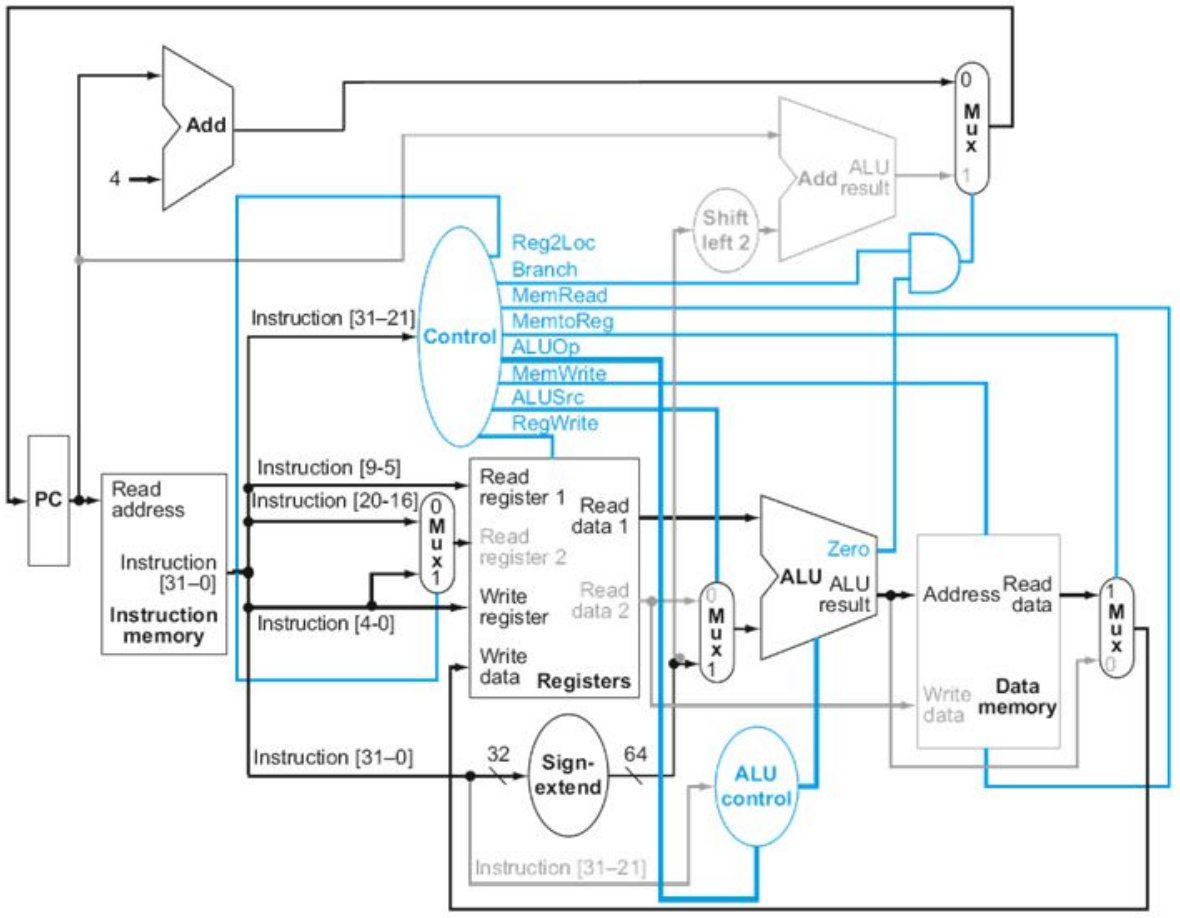
Setting of the control lines

Instruction	Reg2Loc	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	ALUOp1	ALUOp0
R-format	0	0	0	1	0	0	0	1	0
LDUR	X	1	1	1	1	0	0	0	0
STUR	1	1	X	0	0	1	0	0	0
CBZ	1	0	X	0	0	0	1	0	1

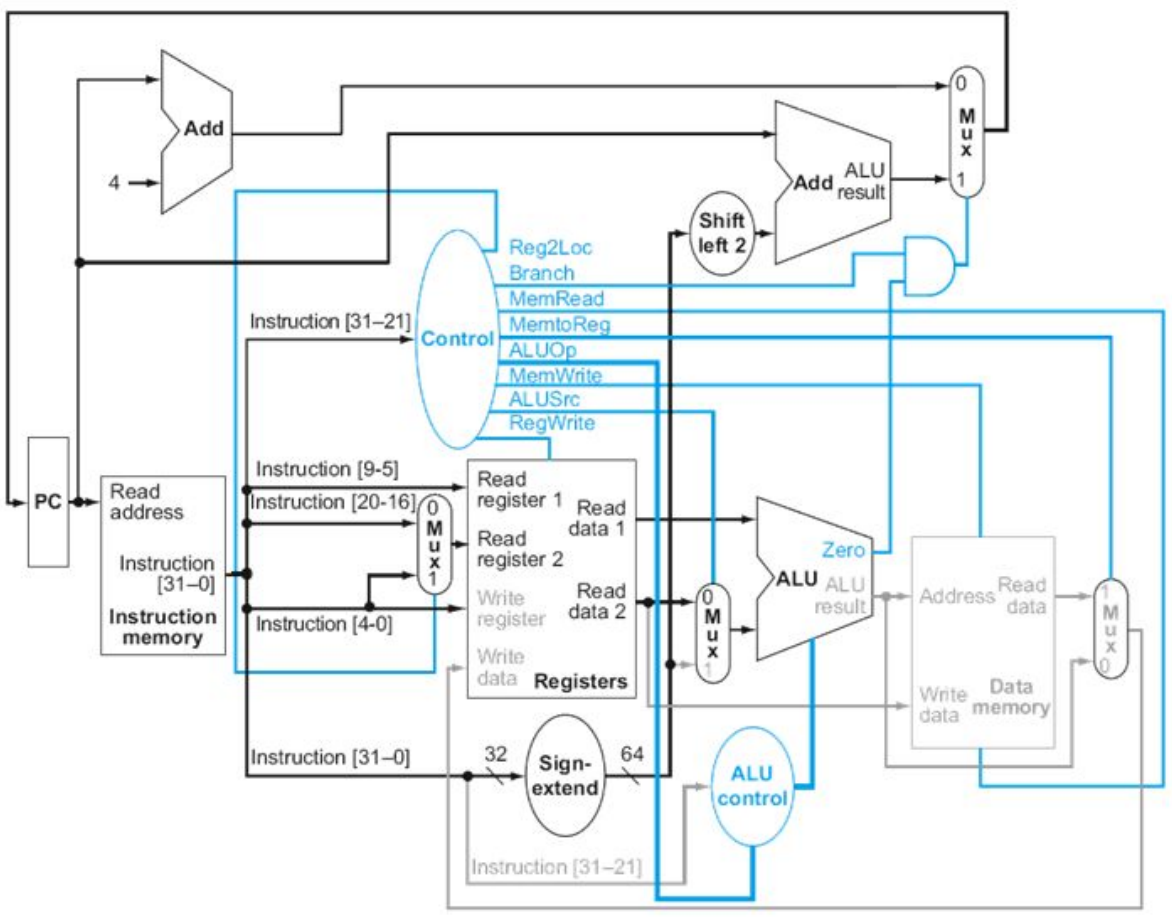
R-Type Instruction



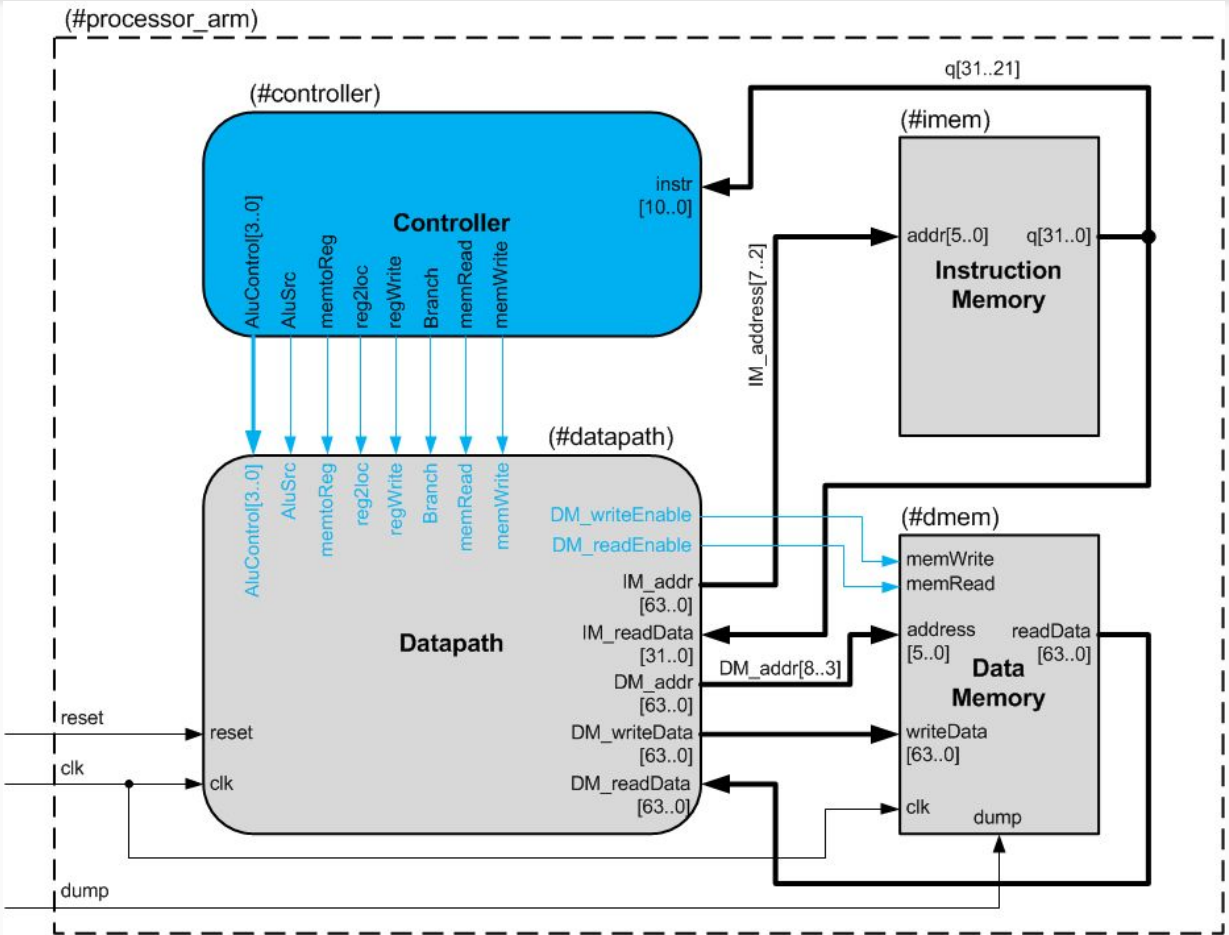
Load Instruction



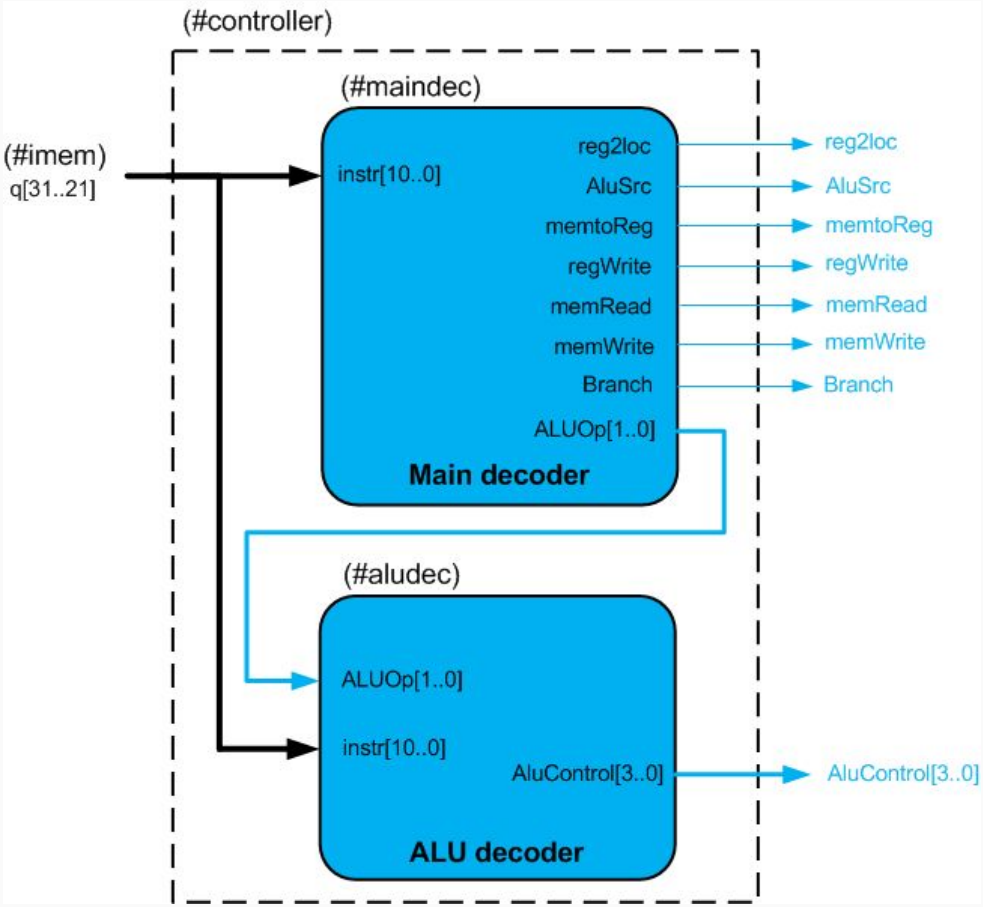
CBZ Instruction



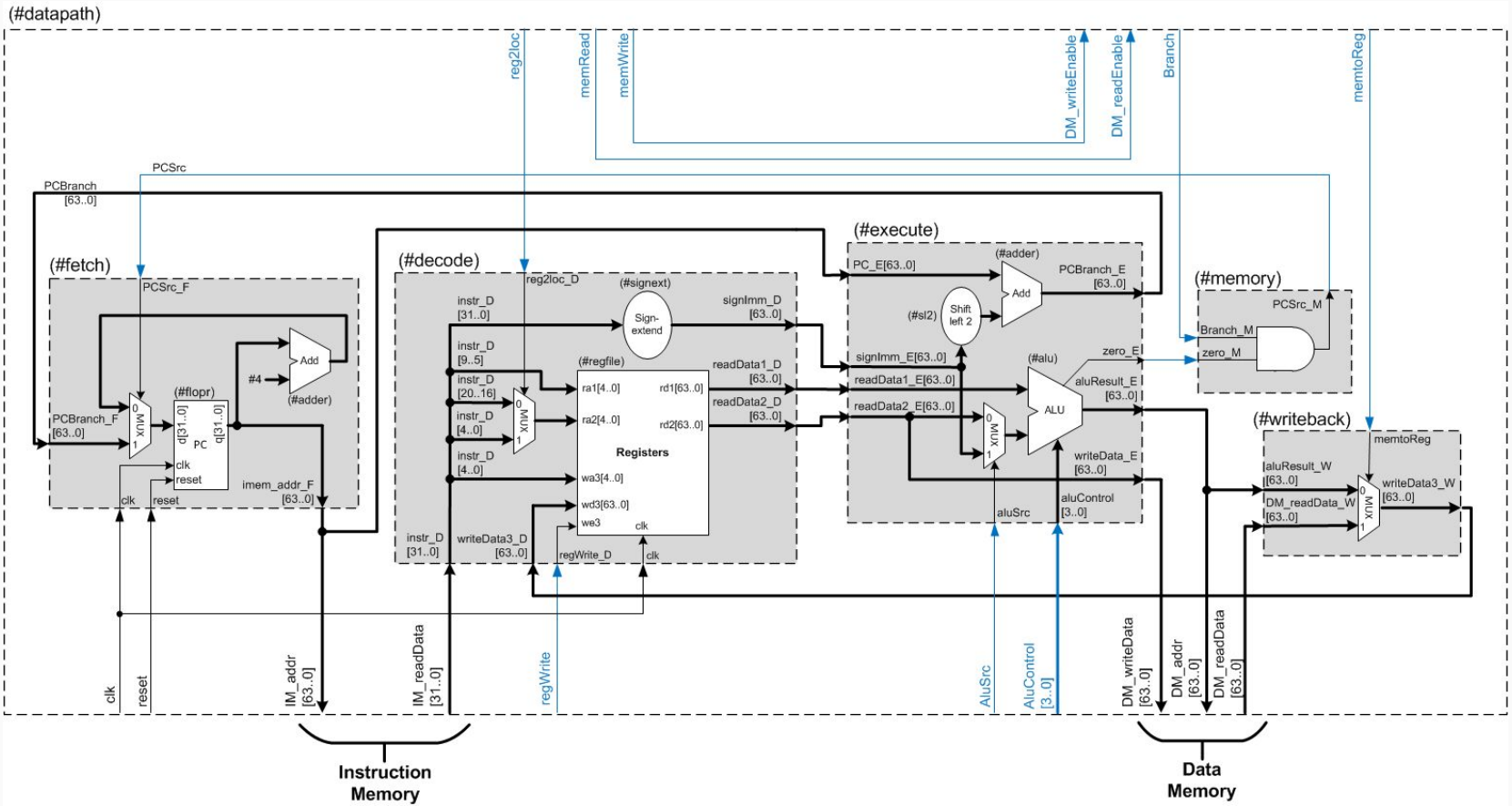
Processor_arm



Controller


(#controller)
(#maindec)
(#imem)
q[31..21]
instr[10..0]
reg2loc
AluSrc
memtoReg
regWrite
memRead
memWrite
Branch
ALUOp[1..0]
Main decoder
reg2loc
AluSrc
memtoReg
regWrite
memRead
memWrite
Branch
(#aludec)
ALUOp[1..0]
instr[10..0]
AluControl[3..0]
ALU decoder
AluControl[3..0]

Datapath


(#datapath)
reg2loc
memRead
memWrite
DM_writeEnable
DM_readEnable
Branch
memtoReg
PCSrc
PCBranch [63..0]
(#fetch)
PCSrc_F
(#flopr)
PC
d[31..0]
q[31..0]
clk
reset
Add
#4
(#adder)
PCBranch_F [63..0]
Mux
imem_addr_F [63..0]
(#decode)
reg2loc_D
(#signext)
Sign-extend
instr_D [31..0]
signImm_D [63..0]
instr_D [9..5]
instr_D [20..16]
instr_D [4..0]
instr_D [4..0]
(#regfile)
ra1[4..0]
ra2[4..0]
rd1[63..0]
rd2[63..0]
Registers
wa3[4..0]
wd3[63..0]
we3
readData1_D [63..0]
readData2_D [63..0]
instr_D [31..0]
writeData3_D [63..0]
regWrite_D
(#execute)
PC_E[63..0]
(#adder)
Add
PCBranch_E [63..0]
(#sl2)
Shift left 2
signImm_E[63..0]
readData1_E[63..0]
readData2_E[63..0]
(#alu)
ALU
zero_E
aluResult_E [63..0]
writeData_E [63..0]
aluControl [3..0]
aluSrc
(#memory)
PCSrc_M
Branch_M
zero_M
(#writeback)
memtoReg
aluResult_W [63..0]
DM_readData_W [63..0]
writeData3_W [63..0]
clk
reset
IM_addr [63..0]
IM_readData [31..0]
regWrite
AluSrc
AluControl [3..0]
DM_writeData [63..0]
DM_addr [63..0]
DM_readData [63..0]
Instruction Memory
Data Memory